TANTGen: Transformer-based Adversarial Network Traffic Generator

INÊS PISSARRA, Instituto Superior Técnico, Portugal

Machine Learning (ML) has made notable advances, with applications across a broad range of fields, including cybersecurity. As new attacks emerge, several ML-based techniques have been studied to detect known and zero-day attacks. Although ML has shown excellent results, it has also demonstrated vulnerabilities to adversarial attacks, where inputs are slightly perturbed to induce misclassification. New research has explored adversarial attack strategies to identify these vulnerabilities and improve model robustness. However, many existing approaches rely on impractical assumptions, such as attacker access to model parameters, or fail to ensure that attacker-induced changes comply with network protocol rules and semantics.

In this thesis, we present TANTGen, a framework to generate realistic adversarial network traffic targeting ML-based Network Intrusion Detection Systems. A key innovation is the Transformer architecture, which excels at modeling complex sequences with intricate dependencies, enabling the capture of relationships between network features to generate effective attacks. TANTGen operates under a black-box threat model, where the attacker has no knowledge of the target model, while preserving protocol semantics. TANTGen achieves attack success rates of up to 99.9% when no perturbation constraints are imposed, and outperforms TrafficManipulator [25] under stricter conditions, achieving an Attack Success Rate of 99.9% against 75.2% under a 6× of the original time constraint and 22.2% against 18.1% under a 3× constraint. By enabling the generation of realistic adversarial attacks, TANTGen provides a foundation for systematically studying model vulnerabilities and improving the robustness of ML-based detection systems.

Keywords: Machine Learning, Adversarial Attack, Intrusion Detection System, Transformer

1 Introduction

Nowadays, Machine Learning (ML) is widely applied across a broad range of domains, from medical diagnostics to the creation of Autonomous Vehicles. With the creation of increasingly flexible and general-purpose models, its use in the computer networking field has also increased. Network Intrusion Detection Systems (NIDSs) are an example of systems that leverage Machine Learning to detect attacks in network environments, but other applications in computer networking also exist, including traffic engineering, routing, resource and fault management, among others [9]. However, as the use of ML continues to expand, researchers have also explored methods to deceive these ML-powered systems by subtly altering their input features in ways that are imperceptible to the monitoring systems (or to the human eye) but can significantly disrupt system performance. These adversarial attacks have demonstrated critical vulnerabilities in the application of Machine Learning across many domains. Exploring these vulnerabilities is relevant, as it enables the evaluation and improvements of defensive mechanisms, ultimately enhancing their robustness and reliability against real-world threats.

Adversarial attacks have been widely studied in domains such as computer vision [7, 11, 22], where attackers manipulate image pixel values to deceive classifiers. In this context, generating adversarial examples is relatively straightforward because the features used by the model are the pixel values themselves, which are differentiable and can be easily adjusted without violating any constraints. However, extending existing techniques to network traffic [8, 18, 25–27, 32, 40, 43, 47, 55] presents several challenges. Since network traffic is often encrypted, the features extracted to feed ML models are typically based on characteristics such as packet headers, packet sizes, interarrival patterns, and other forms of network metadata. Unlike image pixels, these features are derived rather than directly observed, which introduces the **inverse feature mapping problem**: adversarial modifications applied in the feature space must be accurately translated back into realizable network traffic while preserving protocol compliance and semantic correctness.

Some studies [18, 43, 55] bypass this challenge by applying modifications directly to the extracted features and feeding them to the target model. This scenario is unrealistic, as it assumes a strong adversary with access to the internal feature extraction and model pipeline. In contrast, more recent work [8, 25–27, 32, 40, 47] explicitly considers the inverse feature mapping problem and applies modifications directly to network traffic. While more realistic, these approaches still exhibit important limitations, particularly in terms of scalability, efficiency, or the ability to preserve traffic semantics.

Another unrealistic assumption made in several studies [26, 27, 40, 43, 55] is that the adversary has full knowledge of the model and its parameters, allowing the generation of attacks specifically tailored to the target model — a *white-box* attack. Black-box attacks, where the adversary has no prior knowledge of the model being used [8, 18, 25, 32, 47, 55], are generally considered more realistic and, due to their ability to transfer across models, may pose a greater practical threat when effective. In this study, **we focus on black-box attacks** as they better reflect real-world adversarial conditions, where attackers have little to no knowledge of the underlying models.

An additional fundamental challenge lies in preserving the realizability of adversarial traffic, since network protocols impose strict constraints on how traffic may be modified while remaining legitimate. For example, excessive perturbations to packet timing or size can render traffic benign or non-functional, preventing it from achieving the intended malicious objective. The real-time deployment of adversarial traffic aggravates these challenges further: unlike offline attacks, real-time attacks require modifications to be performed on-the-fly under dynamic network conditions, and any additional latency introduced by adversarial sample generation may itself become a signal for detection.

1.1 Rationale and Contributions

This thesis introduces **TANTGen**, a new framework based on the Transformer architecture [49] designed to generate adversarial traffic that adheres to network protocol constraints. Transformers have demonstrated strong capabilities in capturing complex dependencies and contextual relationships in domains such as Natural Language Processing (NLP). Similarly, network traffic exhibits rich temporal and structural dependencies across packets and features, which must be preserved to ensure feasibility under protocol constraints.

Author's Contact Information: Inês Pissarra, ines.pissarra@tecnico.ulisboa.pt, Instituto Superior Técnico, Lisbon, Portugal.

Furthermore, the ability to pre-train Transformer models with large unlabeled datasets is an advantage given the scarcity of labeled adversarial traffic datasets, and the self-attention mechanism enables efficient processing suitable for online or near real-time deployment scenarios.

This thesis makes the following contributions:

- A Transformer-based framework for adversarial traffic generation in the traffic-space domain.
- A black-box threat model that enforces network protocol constraints, ensuring realistic attack scenarios.
- A comprehensive evaluation using real-world datasets and NIDSs, assessing both attack effectiveness and realism.
- An experimental demonstration of effective evasion while preserving traffic semantics. Under a strict 3× flow duration ratio constraint, TANTGen achieves an ASR of 22.2% compared to 18.1% from TrafficManipulator [25], and under a more relaxed 6× constraint, the gap widens significantly, with TANTGen reaching 99.9% against 75.2%.

2 Background

This section provides the necessary background on Machine Learning (ML), Adversarial Attacks on Machine Learning, and ML-based networking systems.

2.1 Machine Learning

Machine learning has gained significant popularity due to its ability to achieve high accuracy across a wide range of tasks. The typical ML pipeline begins with data collection and Exploratory Data Analysis (EDA), followed by preprocessing — which may include handling missing values, normalization, and encoding categorical variables — and feature engineering or automatic representation learning. Well-constructed representations reduce data complexity, improve computational efficiency, and help prevent overfitting.

2.1.1 Model Training and Deployment. Training approaches can be divided into three main categories. In **Supervised Learning**, models learn the relationship between input features and labels, encompassing models such as Decision Trees [28], Neural Networks [34], Support Vector Machines [12], and Logistic Regression [13]. In **Unsupervised Learning**, only samples are provided, and the model groups the data without explicit labels; examples include GANs [23] and VAEs [29]. In **Reinforcement Learning**, an agent interacts with an environment, learning a mapping between states, actions, and rewards to maximize long-term cumulative reward [50].

Fine-tuning pre-trained models is a practical strategy that requires significantly less data and computational resources compared to training from scratch, and is especially beneficial when dealing with limited data.

Generative Adversarial Networks (GANs) [23] consist of two competing neural networks: a Generator that produces new data instances and a Discriminator that evaluates their authenticity. GANs have been applied in networking, for example in NetShare [52], to generate synthetic traffic traces that preserve statistical properties of real traffic.

Variational Autoencoders (VAEs) [29] are probabilistic generative models consisting of an encoder that maps input data into a probabilistic latent space, characterized by a mean vector μ and standard deviation vector σ, and a decoder that reconstructs the data from a sampled latent variable z. This framework balances reconstruction quality and latent space regularization.

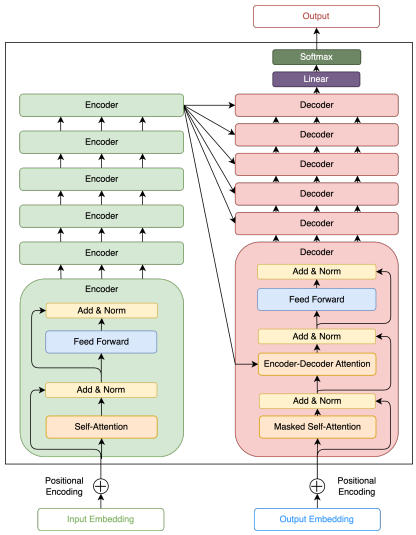


Fig. 1. Transformer Architecture

The **Transformer** [49] is an architecture based on self-attention mechanisms that model how different parts of the input relate to each other, capturing contextual information and long-range dependencies. This is particularly important for network traffic, where dependencies across packets and time must be preserved. Transformers process inputs in parallel, enabling efficient training and making them well-suited for long-sequence modeling. Depending on the architecture, a Transformer may include an encoder, a decoder, or both. Encoder-only models such as BERT [16] are optimized for classification; decoder-only models such as GPT [41] for generative tasks; and encoder-decoder models such as T5 [42] and BART [31] for tasks like translation and summarization. Figure 1 illustrates the general Transformer architecture.

2.2 Adversarial Attacks on Machine Learning

2.2.1 Types of Adversarial Attacks. Adversarial attacks introduce small, carefully crafted perturbations to the input in order to induce misclassification, while remaining imperceptible to the human eye or monitoring system, and may target different stages of the model lifecycle. **Poisoning attacks** [6, 44, 51] inject malicious samples into the training set to degrade accuracy or introduce vulnerabilities. **Evasion attacks** [18, 25–27, 32, 43, 47, 55] craft inputs to trick the model into misclassifying them at deployment time.

Attacks are further categorized by the attacker's knowledge. **White-box attacks** [26, 27, 40, 43, 55] assume complete knowledge of the target model, achieving high success rates but under unrealistic assumptions. **Gray-box attacks** [25] involve partial knowledge, such as access to the model architecture but not its parameters. **Black-box attacks** [8, 18, 25, 32, 38, 47, 55] assume

no direct knowledge of the target model, making them more realistic. These are further divided into **query-based attacks** [8, 32], where the attacker iteratively queries the model and adjusts inputs based on observed outputs, and **transfer-based attacks** [25, 38], where adversarial examples crafted against a surrogate model are transferred to the target, exploiting the transferability property first observed by Szegedy et al. [48].

2.2.2 Defenses against Adversarial Attacks. Several defense mechanisms have been proposed to improve model robustness. **Adversarial training** [24, 48] exposes models to adversarial examples during training, enabling them to recognize and resist such perturbations. **Feature design and adjustment** [15, 21] reduces the attack surface through insertion, deletion, and resizing of features. **Sanitization** [14, 46] identifies and removes malicious inputs before they affect the model. **Defensive distillation** [38] smooths the model's decision surface by training a student network on the soft-label outputs of a teacher network, improving generalization. Finally, **ensemble models** [4, 5] leverage the strengths of multiple models to mitigate individual biases and improve robustness.

2.3 ML-based Networks

Network traffic features are typically classified into **packet-based features**, extracted from individual packets (e.g., packet size, header information, payload, inter-arrival time), and **flow-based features**, derived from groups of packets sharing the same flow (e.g., flow duration, number of packets or bytes exchanged).

ML has been applied across many networking domains [9], including traffic prediction [53], traffic classification [37], routing optimization [20], congestion control [33], resource and fault management [39], and QoS/QoE management [36].

In **network security**, ML plays a crucial role in detecting threats such as intrusion and Denial-of-Service (DoS) attacks. **Network Intrusion Detection Systems (NIDSs)** inspect packets and flows to detect malicious activity. **Signature-based NIDS** compares traffic patterns against known attacks, being effective but unable to detect zero-day threats. **Anomaly-based NIDS** identifies deviations from normal behavior, enabling zero-day detection at the cost of higher false positive rates. Kitsune [35], for example, uses an ensemble of autoencoders trained on benign traffic to identify anomalies in real-time through reconstruction loss.

3 Related Work

This section reviews the state-of-the-art on adversarial attacks in ML-based network systems and related uses of Transformer architectures, highlighting the limitations that motivate TANTGen.

3.1 Adversarial Attacks in ML-based Networks

Adversarial attacks in ML-based networks can be categorized along two orthogonal dimensions: the space in which the attack is performed (feature-space vs. traffic-space), and the attacker's knowledge of the target model (white-box vs. black-box). **Feature-space attacks** directly manipulate the input features of ML models, but are unrealistic as they require access to the feature extraction pipeline and suffer from the inverse feature mapping problem. **Traffic-space attacks** apply modifications directly to packets — typically in timing and size — while respecting protocol constraints, making them more realistic. Figure 2 illustrates these four attacker types.

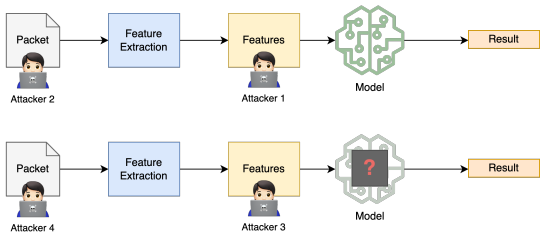


Fig. 2. Attacker types

3.1.1 White-box Feature-space Attacks. This is the attack performed by Attacker 1, which makes the least realistic assumptions. Here, the adversary is positioned after feature extraction, modifying and sending the features directly to the target model.

Zolbayar et al. [55] proposed NIDSGAN, which uses a Wasserstein GAN (WGAN) to generate adversarial traffic flows against NIDSs. The generator mutates malicious traffic to minimize perturbations while evading detection, and the discriminator enforces indistinguishability from real attack flows. Inherent constraints and valid feature ranges are enforced to maintain traffic realizability.

Venkata et al. [43] used VAEs to reconstruct malicious packets, exploring other variants for better approximation of non-linear functions and to increase sample diversity.

3.1.2 White-box Traffic-space Attacks. This is the attack represented by attacker 2. Here, the attacker modifies real network traffic and is therefore more realistic than the previous case.

Hashemi et al. [26] propose three packet-level mutations targeting NIDSs: splitting packets into smaller ones, adjusting inter-packet delays to mimic benign patterns, and injecting extra packets. Mutations are applied iteratively to reduce the malicious classification probability, but this process is computationally expensive and unsuitable for real-time deployment.

Pierazzi et al. [40] introduced the formalization of problem-space (i.e., traffic-space) attacks, defining transformations that preserve semantics, plausibility, and robustness to preprocessing. Although targeting malware classifiers, the framework is broadly applicable and relevant to our problem.

Jin et al. [27] proposed PANTS, combining Adversarial Machine Learning techniques (PGD or ZOO) with Satisfiability Modulo Theories solvers to find realizable packet sequences consistent with domain constraints. It considers two attacker positions: end-host attackers, who can delay, inject, and pad packets, and in-path attackers, restricted to delays only. However, its iterative process is also not suitable for real-time deployment.

3.1.3 Black-box Feature-space Attacks. We now turn to attacker 3. These attacks inherit the limitations of feature-space approaches (notably inverse feature mapping) while operating under more realistic threat assumptions, where the attacker does not know the model being attacked.

Duy et al. [18] proposed DIGFuPAS, combining a WGAN with function-preserving feature manipulation. A surrogate model is trained on the target model's outputs and used to guide adversarial generation. The framework distinguishes between functional and non-functional features, keeping the former unchanged to preserve attack effectiveness.

Zolbayar et al. [55] extend their white-box NIDSGAN to the black-box setting by incorporating a surrogate model trained via queries to the target. They also introduce a restricted black-box setting with limited query access and an active learning approach to refine the surrogate model near the decision boundary.

3.1.4 Black-box Traffic-space Attacks. Finally, for an attack made by attacker 4, no prior knowledge about the attacked model is needed, and it operates directly on network traffic. Attacker 4 represents the most realistic and dangerous type of attacker.

Han et al. [25] used a GAN to generate adversarial features and a Particle Swarm Optimization (PSO) algorithm to mutate network traffic accordingly. The GAN generator transforms malicious features into adversarial versions, while the discriminator acts as a surrogate model. PSO then determines how to modify actual traffic — by adding delays or injecting packets — to reproduce those features while respecting overhead and timing constraints. A direct comparison between TANTGen and this approach is presented in Section 5.

Sun et al. [47] proposed GPMT, which uses a WGAN to generate adversarial traffic by adjusting packet timestamps and payload lengths. A probing phase builds a surrogate model from target model outputs, which is then used as the WGAN discriminator. However, the magnitude constraints applied do not explicitly account for protocol-specific restrictions at the application layer, potentially producing malformed packets.

A key limitation shared by GAN-based approaches [25, 47] is their inability to effectively capture long-range dependencies across packets. This motivates the use of Transformer architectures, which are inherently designed for sequential dependency modeling.

In the censorship evasion domain, Bock et al. [8] proposed Geneva, which uses an evolutionary algorithm to discover packet manipulation strategies (duplication, fragmentation, tampering, dropping) that bypass censorship classifiers. Its iterative nature makes it slow and more prone to detection.

Liu et al. [32] created Amoeba, a reinforcement learning agent that perturbs packets via truncation, padding, and delay to evade network censors. Despite operating in a black-box setting and handling noisy rewards, Amoeba tends to exploit a single vulnerability rather than exploring diverse strategies, limiting its effectiveness for adversarial training [27].

3.2 Use of Transformers for Networking Tasks

Several studies have explored pre-trained Transformer models for networking tasks. Le et al. [30] examined the applicability of BERT and GPT-3 to network traffic analysis, noting structural similarities between network traffic and natural language and arguing that the abundance of unlabeled traffic data supports a foundational model pre-training.

Table 1. Comparison between existing research

Reference	Year	Attack Type	Attack Scope	Model
B.-E. Zolbayar et al. [55]	2022	❍ & ●	🧩	WGAN
P. Venkata et al. [43]	2023	❍	🧩	VAE w/WNN
M. J. Hashemi et al. [26]	2019	❍	🌐	Own algorithms
F. Pierazzi et al. [40]	2020	❍	🌐	Own algorithms
M. Jin et al. [27]	2024	❍	🌐	PGD & ZOO & SMT
P. T. Duy et al. [18]	2021	●	🧩	WGAN
K. Bock et al. [8]	2019	●	🌐	Own algorithms
H. Liu et al. [32]	2023	●	🌐	RL
D. Han et al. [25]	2021	●	🌐	GAN & PSO
P. Sun et al. [47]	2023	●	🌐	WGAN
Our thesis	2025	●	🌐	Transformer

Legend: ❍ White-box attack, ● Black-box attack, 🧩 Feature-space scope, 🌐 Traffic-space scope
GAN - Generative Adversarial Network ; WGAN - Wasserstein Generative Adversarial Network ; VAE - Variational Autoencoder ; WNN - Wavelet Neural Networks ; PGD - Projected Gradient Descent ; ZOO - Zeroth-order Optimization ; SMT - Satisfiability Modulo Theories ; RL - Reinforcement Learning; PSO - Particle Swarm Optimization

Dietmüller et al. [17] proposed the Network Traffic Transformer (NTT), an encoder-only transformer that incorporates packet-level features into embeddings with hierarchical aggregation for long- and short-term dependencies. Pre-trained to predict end-to-end delay, NTT can be fine-tuned for multiple downstream tasks.

Zhou et al. [54] proposed NetFlowGen, a decoder-only Transformer pre-trained to predict the next step of network traffic from NetFlow records. A key contribution is the discretization of continuous features using a binning algorithm, addressing the mismatch between continuous network features and the discrete token vocabularies typical of NLP Transformers.

Motivated by these works, this thesis applies Transformer architectures to adversarial traffic generation under realistic constraints, combining a black-box threat model, traffic-space modifications, and protocol-compliant generation — a combination not previously explored.

3.3 Summary

Table 1 summarizes existing approaches and highlights the gap addressed by this thesis. Overall, existing methods either rely on unrealistic assumptions (feature-space or white-box), fail to preserve traffic semantics, or struggle to capture long-range dependencies. These limitations motivate the design of TANTGen as a Transformer-based, black-box, traffic-space adversarial generation framework.

4 Solution

This section presents TANTGen, a novel Transformer-based architecture for generating adversarial traffic aimed at fooling ML-based NIDSs. We describe the threat model, the architecture, and the implementation.

4.1 Threat Model

The considered threat model addresses evasion attacks on ML-based NIDSs operating in the traffic domain. The adversary's objective is to introduce minimal modifications directly to network packets — manipulating timing and size — such that the NIDS misclassifies malicious traffic as benign. Unlike feature-space attacks, the adversary operates on real network traffic without access to the internal feature extraction pipeline.

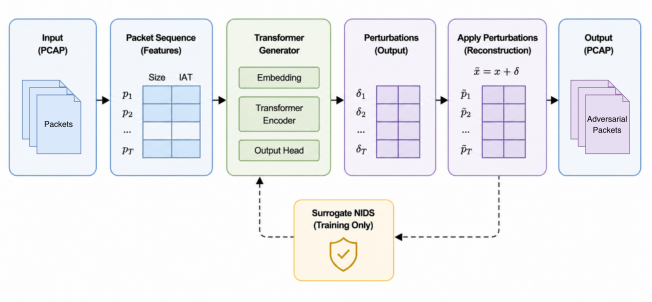


Fig. 3. TANTGen Overview

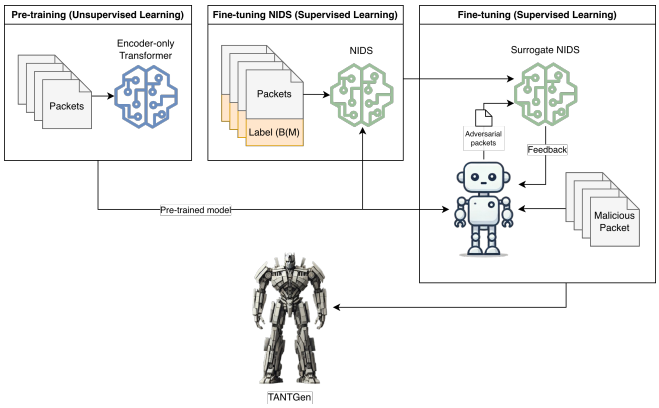


Fig. 4. TANTGen training process

We assume a **black-box** setting, where the adversary has no knowledge of the model's architecture, parameters, or training data. We further assume an **end-host adversary** capable of generating and shaping its own traffic, enabling fine-grained control over packet timing and payload structure.

Temporal manipulations (e.g., inter-packet delays) must remain small to avoid degrading attack effectiveness, raising suspicion, or interfering with protocol dynamics such as TCP retransmission timeouts. Spatial manipulations (e.g., padding) must preserve protocol correctness, requiring consistent updates to dependent fields such as length indicators and checksums. Although perturbations target packet-level features, they are always mapped back to valid traffic, ensuring realizability. Under these constraints, the adversary seeks to achieve successful evasion with minimal perturbation cost.

4.2 TANTGen Architecture Overview

This section presents the proposed transformer-based architecture for generating adversarial network traffic, covering the data representation and problem formulation, the rationale behind model selection, the training methodology, and the attack application process.

Figure 3 provides a high-level overview of TANTGen, illustrating its main components and their interactions.

4.2.1 Data Representation and Problem Formulation. Inspired by prior work such as Amoeba [32] and PANTS [27], traffic is represented as a sequence of packets based on two fundamental attributes: *packet size* and *inter-arrival time (IAT)*. These features capture the observable characteristics exploited by traffic analysis-based NIDSs, even in encrypted settings. Formally, a flow is represented as $[p_1, p_2, \dots]$, where each packet $p_t = [\text{Packet Size}_t, \text{IAT}_t]$.

Preprocessing is kept minimal to preserve signal fidelity. IAT values are clipped to be non-negative and log-transformed ($\log(1 + x)$) to reduce distribution skewness, which is particularly important given the heavy-tailed nature of inter-arrival times [10, 19]. Both features are then z-score standardized using training set statistics. Sequences are partitioned into fixed-length windows via a sliding window mechanism with configurable stride, and each packet is projected into a higher-dimensional latent space via a learnable linear projection layer.

Given an input sequence $[p_1, \dots, p_T]$, the model outputs a sequence of perturbation vectors $[\delta_1, \dots, \delta_T]$, where each

$$\delta_t = [\Delta\text{Packet Size}_t, \Delta\text{IAT}_t]$$

is applied to the corresponding packet to obtain the adversarial sequence $\tilde{x}$.

4.2.2 Model Selection. The task is formulated as a **sequence-to-sequence regression problem with positional alignment**: given an input packet sequence, the model produces a position-wise perturbation vector of the same length. This formulation favors **encoder-only** Transformer architectures, which process the full input context through self-attention layers and produce aligned output representations — in contrast to encoder-decoder models suited for variable-length outputs or decoder-only models for autoregressive generation. The contextual representations produced by the encoder are mapped through a feed-forward output head to generate per-packet perturbations.

4.2.3 Transformer Training. TANTGen training follows a two-phase self-supervised approach, as illustrated in Figure 4.

In the **pre-training phase**, the model is trained on unlabeled traffic using a masked reconstruction objective: a random subset of packet features is masked, and the model reconstructs the original values from the surrounding context. Small stochastic noise is injected during training to encourage robustness. This enables the model to capture both short-term dependencies and global traffic patterns, providing a strong initialization for fine-tuning.

In the **fine-tuning phase**, a surrogate NIDS is first constructed by attaching a classification head to the frozen pre-trained encoder and training it in a supervised manner to classify traffic as benign or malicious. Once trained, its parameters are frozen. The pre-trained transformer is then unfrozen and fine-tuned to produce adversarial perturbations under a composite objective:

$$\mathcal{L}_{\text{total}}(x, \delta) = \lambda_{\text{e}}\, \mathcal{L}_{\text{evasion}}(\mathcal{R}(x, \delta)) + \lambda_{\text{p}}\, \mathcal{L}_{\text{pert}}(\delta), \tag{1}$$

where the loss terms are defined as:

$$\mathcal{L}_{\text{evasion}}(\tilde{x}) = \text{mean}\left(\max(0, f_\psi(\tilde{x}) - m)\right), \tag{2}$$

$$\mathcal{L}_{\text{pert}}(\delta) = \text{mean}\left(\delta_{\text{IAT}}\right) + \alpha \cdot \text{mean}\left(\delta^2_{\text{size}}\right). \tag{3}$$

Notation.

- x Original packet sequence.
- $\tilde{x} = \mathcal{R}(x, \delta)$ Adversarial sequence after applying perturbation δ with validity constraints.
- λ_e, λ_p Loss weights for evasion and perturbation terms.
- f_ψ Surrogate NIDS model.
- m Margin parameter.
- α Hyperparameter balancing packet size perturbation magnitude.

A linear penalty is used for IAT to capture the cumulative cost of added delay, while a quadratic penalty is used for packet size to more strongly penalize large deviations and promote realism.

4.2.4 Attack Application. After training, the generator produces adversarial traffic directly in the traffic domain. Given a malicious input sequence x, the model outputs perturbations δ that are applied and discretized to ensure realizability. Several protocol-aware constraints are enforced during reconstruction: a conservative MTU threshold caps packet sizes below 1500 bytes; packets belonging to security-sensitive protocols (SSH, TLS, SMB, RDP) are left unmodified to avoid breaking encryption or authentication handshakes; and for plaintext protocols, padding is introduced in a protocol-compliant manner (e.g., inserting optional header fields in SSDP messages). IP and TCP checksums are recomputed in all cases.

The result is a valid PCAP file that preserves the structure and semantics of the original flow while incorporating adversarial modifications, ready to be replayed against the target NIDS in a fully black-box traffic-space setting.

4.3 Implementation

TANTGen was implemented in `Python` (v3.10.12) following a modular design aligned with the architecture described in the previous sections. Traffic preprocessing and feature extraction used `Pandas` and `NumPy`, while `Scapy` handled PCAP parsing, manipulation, and reconstruction. The transformer model was implemented in `PyTorch` and trained with `PyTorch Lightning`.

The NTT framework [17] was used as the starting point, given its alignment with key requirements: raw packet-level representation, sequential processing without handcrafted features, and a well-structured encoder architecture. However, NTT required substantial adaptation: the pre-training objective was replaced with a masked reconstruction strategy over both packet size and IAT; a surrogate NIDS was developed by repurposing the pre-trained encoder as a binary classifier; and the fine-tuning loop was redesigned around adversarial optimization with a frozen surrogate providing evasion feedback.

5 Evaluation

This section evaluates TANTGen's ability to generate adversarial network traffic that evades ML-based NIDSs while remaining protocol-compliant and functionally malicious.

5.1 Experimental Settings

5.1.1 Target NIDSs and Feature Extractors. Several supervised ML-based NIDSs were evaluated: **LR**, **MLP**, **RF**, **SVM**, and **CNN**. These models use **CICFlowMeter** [1, 2] for flow-based feature extraction, aggregating packets into bidirectional flows and computing statistical features such as flow duration, packet counts, byte rates, and inter-arrival times.

Additionally, **Kitsune** [35] was employed as an anomaly-based NIDS that uses an ensemble of autoencoders and is trained exclusively on benign traffic. We adopt the version proposed by Han et al. [25] for faster execution and direct comparability. Kitsune uses the **AfterImage** [35] packet-based feature extractor, which incrementally computes statistical features over multiple temporal windows, making it inherently more sensitive to timing perturbations than flow-based approaches.

5.1.2 Datasets. Two publicly available datasets were used. The **CIC-IDS2017** dataset [45] contains realistic benign and attack traffic. Benign traffic was used for pre-training. Three attack categories met the minimum threshold of 10,000 flows for fine-tuning and evaluation: **Brute Force** (13,835 flows), **DDoS** (128,027 flows), and **Port Scan** (158,930 flows).

The **Kitsune dataset** [3] contains benign and malicious IoT traffic. Two attacks met the minimum threshold of 500,000 packets and showed a sufficiently high initial detection rate: **Mirai Botnet** (642,516 packets) and **SSDP Flood** (1,439,604 packets).

Each dataset was partitioned into four subsets: target training set (benign only for Kitsune), surrogate model set, adversarial training set (malicious only), and test set. Class balancing was applied through random undersampling where applicable.

5.1.3 Attacker's Capability. Two perturbation types are considered: **packet size modification** (payload padding) and **IAT modification** (inter-packet delays). Both are constrained to preserve attack semantics and protocol compliance. Size restrictions limit the fraction of packets eligible for padding (100%, 50%, or 20%). IAT restrictions cap the resulting flow duration to at most x times the original.

5.1.4 Baseline and Metrics. The primary baseline is the original unmodified malicious traffic. For Mirai Botnet against Kitsune, a comparison with TrafficManipulator [25] is also included. The main evaluation metric is the **Attack Success Rate (ASR)** — the fraction of adversarial samples that successfully evade detection:

$$\text{ASR} = \frac{\text{Number of adversarial samples that evade detection}}{\text{Total number of adversarial samples}} \tag{4}$$

Baseline detection performance is assessed using F1 score, Precision, and Accuracy. Operational feasibility is measured through throughput (packets/s), latency (ms/pkt), and resource usage (CPU, RAM, GPU).

5.2 Results

5.2.1 Adversarial Results on CIC-IDS2017. All target models achieve high F1 scores for DDoS and Port Scan. For Brute Force, performance is lower — particularly for LR (0.636), MLP (0.699), and SVM (0.728) — likely due to the smaller number of available flows, which should

Table 2. Attack Success Rate results on CIC-IDS Dataset

Attack	Model	Feature / Pkt. limit	Original	IAT	Size 100%	Size 50%	Size 20%	Both 100%	Both 50%	Both 20%
Brute Force	CNN		0.025	0.012	0.348	0.050	0.228	0.348	0.025	0.025
	LR		0.037	0.400	1.000	0.400	0.522	1.000	0.537	0.400
	MLP		0.000	0.037	1.000	0.325	0.337	1.000	0.050	0.050
	RF		0.000	0.037	0.033	0.050	0.043	0.033	0.050	0.050
	SVM		0.012	0.400	1.000	0.650	0.554	1.000	0.425	0.400
DDoS	CNN		0.030	0.032	0.141	0.034	0.032	0.807	0.035	0.029
	LR		0.047	0.050	0.360	0.168	0.045	1.000	0.045	0.042
	MLP		0.055	0.059	0.360	0.174	0.055	1.000	0.052	0.051
	RF		0.006	0.006	1.000	0.997	0.884	0.999	0.999	0.879
	SVM		0.011	0.014	0.356	0.134	0.010	1.000	0.005	0.004
Port Scan	CNN		0.001	0.275	0.997	0.826	0.374	1.000	0.875	0.543
	LR		0.000	0.000	1.000	0.175	0.005	1.000	0.170	0.003
	MLP		0.000	0.004	1.000	0.815	0.376	1.000	0.816	0.376
	RF		0.000	0.002	1.000	0.831	0.376	1.000	0.830	0.375
	SVM		0.000	0.000	0.937	0.515	0.349	1.000	0.798	0.373

Colour scale (ASR): Below baseline; ≈ baseline; 0.1 – 0.3; 0.3 – 0.6; 0.6 – 0.95; ≥ 0.95

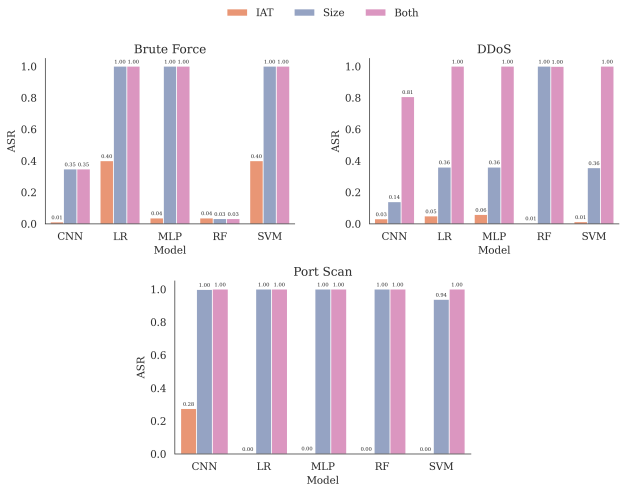


Fig. 5. Attack Success Rate on CIC-IDS dataset

be considered when interpreting adversarial results for this attack type.

Table 2 presents the full adversarial results. IAT-only restricted columns are omitted as the model applies negligible IAT perturbations on this dataset (effective ratios between 1.002× and 1.01×), a behaviour discussed below. Figure 5 illustrates the unrestricted results for a clearer overview.

Impact of size perturbation. Size perturbation consistently proves to be the most effective strategy across all attack types and models in the CIC-IDS setting. In the unrestricted setting (column Size 100%), it achieves ASR of 1.000 against most models for Port Scan, with strong results also for Brute Force and DDoS. This indicates that flow-based features extracted by CICFlowMeter are highly sensitive to byte-count statistics. Port Scan is the most susceptible attack, with near-perfect ASR across all models.

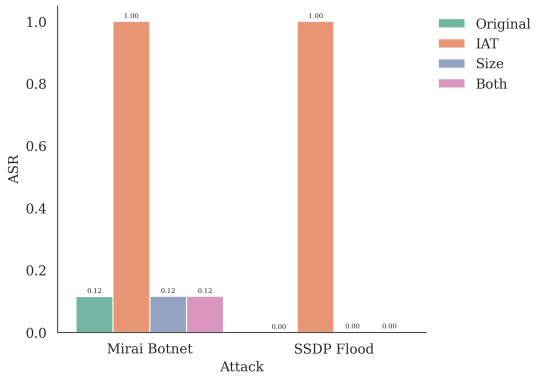


Fig. 6. Attack Success Rate on Kitsune dataset

Impact of IAT perturbation. IAT perturbation alone produces modest evasion gains, as the model spontaneously applies only minimal modifications. This is a consequence of the black-box threat model: since TANTGen optimizes against a packet-level surrogate rather than the flow-level target models, small timing changes appear sufficient to fool the surrogate while keeping the perturbation penalty low but the same doesn't apply to the target model. Forced IAT perturbation experiments confirm that higher IAT budgets do improve ASR — Brute Force is sensitive to modest ratios of around 1.25×, DDoS requires substantially higher ratios, and Port Scan plateaus at around 0.5 for most models.

Impact of combined perturbation. Combining size and IAT perturbations does not consistently outperform size-only modifications, as the model distributes the perturbation budget across two strategies in a way that may be suboptimal against flow-level targets.

Impact of size constraints. Reducing the fraction of packets eligible for size modification degrades ASR progressively, with the impact varying across attack types: Port Scan remains highly susceptible even at 20%, while RF on DDoS achieves near-perfect evasion even at this level; Brute Force shows irregular behaviour, consistent with the difficulty of achieving reliable evasion for this attack type.

Overall, adversarial effectiveness in the CIC-IDS2017 setting is primarily driven by the mismatch between packet-level perturbations and flow-level feature aggregation, highlighting the importance of aligning the adversarial generation process with the feature representation of the target NIDS.

5.2.2 Adversarial Results on Kitsune. Kitsune achieves high baseline F1 scores of 0.939 for Mirai Botnet and 0.998 for SSDP Flood, confirming reliable detection before any perturbation. Tables 3 and 4 present the full adversarial results. Figure 6 illustrates the unrestricted ASR for both attacks.

Impact of size perturbation. Unlike the CIC-IDS2017 results, size perturbation has virtually no effect on Kitsune's detection rate for either attack. This highlights a fundamental difference between flow-based and packet-based detection: Kitsune's AfterImage representation captures fine-grained temporal dynamics, making it largely insensitive to payload size manipulations.

Table 3. Mirai Botnet's ASR results against Kitsune

Feature	Time Ratio	Attack	Mirai Botnet
Original			0.116
IAT	5.48		0.133
	6.4		0.160
	7.25		0.560
	7.59		1.000
	8.38		1.000
Size			0.116
Both			0.116

Table 4. SSDP Flood's ASR results against Kitsune

Feature	Time Ratio	Attack	SSDP Flood
Original			0.000
IAT	3.82		0.002
	20.61		0.185
	32.43		0.419
	51.3		1.000
	63.6		1.000
Size			0.002
Both			0.002

Colour scale (ASR): Below baseline ≈ baseline 0.1 – 0.3 0.3 – 0.6 0.6 – 0.95 ≥ 0.95

Impact of IAT perturbation. IAT perturbation proves to be the decisive factor for evading Kitsune. The ASR increases sharply once the ratio exceeds a threshold, reaching 1.000 for both attacks. Full evasion is achieved at 7.59× for Mirai Botnet and 51.3× for SSDP Flood. The large difference between these thresholds reflects the fundamentally different temporal characteristics of the two attacks: Mirai spans 779.28 minutes over 642,516 packets (≈247 pkt/min), while SSDP Flood concentrates 1,439,604 packets into 11.23 minutes (≈25,640 pkt/min)— approximately 104 times more packets per minute. For SSDP Flood, even large multiplicative ratios translate into small absolute timing shifts, requiring more aggressive perturbation to significantly alter Kitsune's temporal features.

Impact of combined perturbation. Combining size and IAT perturbations yields no improvement over either strategy in isolation, remaining near the baseline for both attacks. As in the CIC-IDS setting, simultaneous optimization of both perturbation types leads the model to a suboptimal trade-off, allocating part of the budget to size modifications that have a negligible effect on Kitsune.

Comparison with TrafficManipulator. TANTGen is compared to TrafficManipulator [25] on the Mirai Botnet example from its public repository (70,000 benign packets for training, 10,000 malicious for evaluation), using two time ratio constraints. Figure 7 presents the results. Under the 6× constraint, TrafficManipulator achieves an ASR of 0.752 while TANTGen reaches 0.999 at an effective ratio of 5.57×. Under the stricter 3× constraint, TANTGen achieves 0.222 against TrafficManipulator's 0.181. In both cases, TANTGen outperforms TrafficManipulator while operating within the same constraints, likely due to its end-to-end learning formulation that directly operates in the traffic domain, compared to TrafficManipulator's two-stage pipeline with feature-level generation followed by heuristic packet reconstruction.

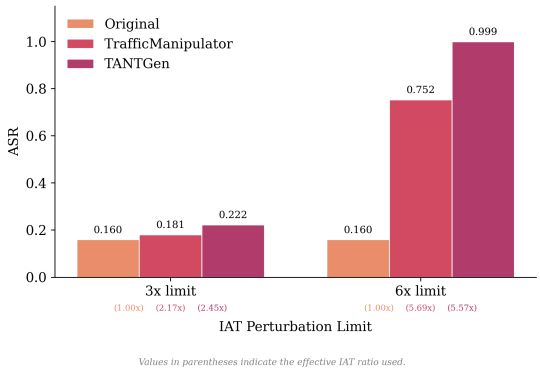


Fig. 7. TANTGen vs TrafficManipulator — Mirai Botnet (Kitsune)

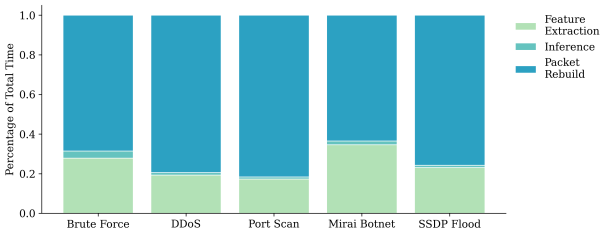


Fig. 8. Relative contribution of each processing stage to total execution time

Taken together, the results on CIC-IDS2017 and Kitsune demonstrate that adversarial effectiveness is tightly coupled to the feature representation used by the target NIDS: flow-based models are highly vulnerable to size manipulations, whereas packet-based models are primarily vulnerable to timing perturbations.

5.2.3 Realism and Maliciousness. Protocol compliance was verified using Wireshark, confirming that modified packets are well-formed and compliant with their protocol specifications, with packet sizes within the MTU threshold and no protocol violations observed. More comprehensive assessments (e.g., statistical comparisons with benign traffic or replay-based validation) are left for future work.

Regarding maliciousness, perturbations pose minimal risk for **Brute Force** (all login attempts are still transmitted) and **Port Scan** (probe packets still reach target ports). For **DDoS** and **Mirai Botnet**, IAT perturbation reduces the packet rate, which may partially degrade flood intensity, though the overall impact is expected to remain significant. The most critical trade-off arises for **SSDP Flood**, where the 51.3× ratio required for full evasion substantially reduces the packet transmission rate, potentially compromising the attack's volumetric effectiveness.

5.2.4 Operational Feasibility. Table 5 summarizes TANTGen's operational performance. The primary bottleneck is packet rebuilding and feature extraction rather than transformer inference, as confirmed by the time breakdown in Figure 8, indicating that the system is predominantly I/O- and CPU-bound.

TANTGen achieves throughput between 743 and 2,039 packets/s with per-packet latencies of 0.49–1.35 ms. IAT-only perturbation

Table 5. End-to-End Computational Cost of TANTGen

Attack	Feature	Throughput (pkt/s)	Latency (ms/pkt)	CPU (%)	RAM (%)	GPU (%)
Brute Force	IAT	1119.0	0.89	288.4	0.57	5.2
	Size	879.9	1.14	304.5	0.59	5.2
	Both	877.6	1.14	308.6	0.59	4.8
DDoS	IAT	1663.2	0.60	188.0	0.57	7.4
	Size	749.6	1.33	196.3	0.60	7.2
	Both	749.0	1.34	186.6	0.60	7.6
Port Scan	IAT	1912.2	0.52	133.5	0.57	10.7
	Size	748.6	1.34	145.9	0.60	8.7
	Both	743.4	1.35	174.5	0.60	9.0
Mirai Botnet	IAT	2038.9	0.49	107.5	0.58	50.2
	Size	1856.6	0.54	107.4	0.61	27.5
	Both	1863.2	0.54	107.3	0.61	38.1
SSDP Flood	IAT	1820.2	0.55	114.4	0.56	59.3
	Size	915.4	1.09	103.2	0.66	47.3
	Both	918.0	1.09	102.9	0.67	48.6

consistently achieves the highest throughput; size perturbation introduces additional overhead from payload insertion and checksum recomputation. RAM utilization remains below 1%, while CPU usage frequently exceeds 100%, reflecting effective multi-core utilization in the feature extraction and packet rebuilding stages. GPU utilization ranges from ≈5% for CIC-IDS2017 attacks to 50–60% for Kitsune attacks, consistent with the larger packet volumes involved.

The achieved throughput remains well below the ≈81,000 pkt/s of a 1 Gbps link, indicating that TANTGen is not suitable for inline real-time deployment in high-throughput environments in its current form. However, it remains applicable for offline attack generation, replay-based evaluation, and lower-rate attack scenarios. Future work could explore kernel bypass options such as Intel DPDK to improve I/O efficiency.

6 Conclusion

This work presented TANTGen, a Transformer-based adversarial traffic generator designed to evaluate the robustness of ML-based NIDSs against evasion attacks. By operating directly in the traffic domain and generating protocol-compliant, functionality-preserving perturbations, TANTGen bridges the gap between traffic-space adversarial attacks and practical evasion strategies.

The evaluation demonstrated that adversarial effectiveness is tightly coupled to the feature representation of the target NIDS. On CIC-IDS2017, size perturbation dominated, achieving perfect or near-perfect ASR against most models for Port Scan, Brute Force, and DDoS. On Kitsune, the situation reversed: size perturbation had virtually no impact, while IAT perturbation proved decisive for both Mirai Botnet and SSDP Flood. This complementary behaviour reveals that robustness cannot be achieved by hardening a single feature space, but requires defenses that jointly exploit multiple, complementary traffic representations. TANTGen also outperformed TrafficManipulator under the same IAT constraints, and protocol compliance was confirmed through Wireshark inspection.

Taken together, these results demonstrate that current ML-based NIDSs remain vulnerable to relatively simple traffic-domain perturbations, and that generating realistic, protocol-compliant adversarial traffic is both feasible and effective in exposing systematic weaknesses in modern network defenses.

6.1 Limitations and Future Work

Despite the promising results, TANTGen has several limitations that motivate future research. First, **maliciousness verification** was assessed only qualitatively; future work should include quantitative validation by replaying adversarial traffic in controlled environments, particularly for high-rate flooding attacks such as SSDP Flood. Second, **feature space expansion** beyond packet size and IAT — incorporating TCP flags, TTL manipulation, packet reordering, or IP header fields — would likely increase evasion effectiveness against a broader range of NIDSs. Third, TANTGen can serve as a tool for **adversarial training**, exposing detection models to evasion examples during training to improve their resilience, and for systematically benchmarking the robustness of different NIDS architectures. Fourth, **real-time deployment** remains a challenge: throughput is currently constrained by packet parsing and reconstruction overhead rather than model inference, and future work should explore high-performance libraries such as Intel DPDK, multiprocessing, and model quantization to enable inline deployment. Finally, integrating **reinforcement learning** into the generation pipeline would allow the model to iteratively refine perturbation strategies through direct interaction with the target NIDS, reducing dependency on labeled data and enabling adaptive attacks that evolve in response to changing detection models.

References

[1] [n. d.]. CICFlowMeter (formerly ISCXFlowMeter). https://www.unb.ca/cic/research/applications.html#CICFlowMeter

[2] [n. d.]. Python version of CICFlowMeter. https://github.com/hieulw/cicflowmeter

[3] 2019. Kitsune Network Attack. UCI Machine Learning Repository. DOI: https://doi.org/10.24432/C5D90Q.

[4] Battista Biggio, Igino Corona, Giorgio Fumera, Giorgio Giacinto, and Fabio Roli. 2011. Bagging classifiers for fighting poisoning attacks in adversarial classification tasks. In *Lecture Notes in Computer Science (including subseries Lecture Notes in Artificial Intelligence and Lecture Notes in Bioinformatics)*, Vol. 6713 LNCS. doi:10.1007/978-3-642-21557-5_37

[5] Battista Biggio, Giorgio Fumera, and Fabio Roli. 2010. Multiple classifier systems for robust classifier design in adversarial environments. *International Journal of Machine Learning and Cybernetics* 1 (2010). Issue 1-4. doi:10.1007/s13042-010-0007-7

[6] Battista Biggio, Blaine Nelson, and Pavel Laskov. 2012. Poisoning attacks against support vector machines. In *Proceedings of the 29th International Conference on Machine Learning, ICML 2012*, Vol. 2.

[7] Battista Biggio and Fabio Roli. 2018. Wild patterns: Ten years after the rise of adversarial machine learning. *Pattern Recognition* 84 (2018). doi:10.1016/j.patcog.2018.07.023

[8] Kevin Bock, Xiao Qiang, George Hughey, and Dave Levin. 2019. Geneva: Evolving censorship evasion strategies. In *Proceedings of the ACM Conference on Computer and Communications Security*. doi:10.1145/3319535.3363189

[9] Raouf Boutaba, Mohammad A. Salahuddin, Noura Limam, Sara Ayoubi, Nashid Shahriar, Felipe Estrada-Solano, and Oscar M. Caicedo. 2018. A comprehensive survey on machine learning for networking: evolution, applications and research opportunities. *Journal of Internet Services and Applications* 9 (2018). Issue 1. doi:10.1186/s13174-018-0087-2

[10] Jin Cao, William S. Cleveland, Dong Lin, and Don X. Sun. 2001. On the nonstationarity of Internet traffic. In *Proceedings of the 2001 ACM SIGMETRICS International Conference on Measurement and Modeling of Computer Systems* (Cambridge, Massachusetts, USA) *(SIGMETRICS '01)*. Association for Computing Machinery, New York, NY, USA, 102–112. doi:10.1145/378420.378440

[11] Antonio Emanuele Cinà, Kathrin Grosse, Ambra Demontis, Sebastiano Vascon, Werner Zellinger, Bernhard A. Moser, Alina Oprea, Battista Biggio, Marcello Pelillo, and Fabio Roli. 2023. Wild Patterns Reloaded: A Survey of Machine Learning Security against Training Data Poisoning. *Comput. Surveys* 55 (2023). Issue 13. doi:10.1145/3585385

[12] Corinna Cortes and Vladimir Vapnik. 1995. Support-Vector Networks. *Machine Learning* 20 (1995). Issue 3. doi:10.1023/A:1022627411411

[13] D. R. Cox. 1959. The Regression Analysis of Binary Sequences. *Journal of the Royal Statistical Society: Series B (Methodological)* 21, 1 (1959), 238–238. doi:10.1111/j.2517-6161.1959.tb00334.x arXiv:https://rss.onlinelibrary.wiley.com/doi/pdf/10.1111/j.2517-6161.1959.tb00334.x

[14] Gabriela F. Cretu, Angelos Stavrou, Michael E. Locasto, Salvatore J. Stolfo, and Angelos D. Keromytis. 2008. Casting out demons: Sanitizing training data for anomaly sensors. In *Proceedings - IEEE Symposium on Security and Privacy*. doi:10.1109/SP.2008.11

[15] Ofer Dekel, Ohad Shamir, and Lin Xiao. 2010. Learning to classify with missing and corrupted features. *Machine Learning* 81 (2010). Issue 2. doi:10.1007/s10994-009-5124-8

[16] Jacob Devlin, Ming Wei Chang, Kenton Lee, and Kristina Toutanova. 2019. BERT: Pre-training of deep bidirectional transformers for language understanding. In *NAACL HLT 2019 - 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies - Proceedings of the Conference*, Vol. 1.

[17] Alexander Dietmüller, Siddhant Ray, Romain Jacob, and Laurent Vanbever. 2022. A new hope for network model generalization. In *HotNets 2022 - Proceedings of the 2022 21st ACM Workshop on Hot Topics in Networks*. doi:10.1145/3563766.3564104

[18] Phan The Duy, Le Khac Tien, Nghi Hoang Khoa, Do Thi Thu Hien, Anh Gia Tuan Nguyen, and Van Hau Pham. 2021. DIGFuPAS: Deceive IDS with GAN and function-preserving on adversarial samples in SDN-enabled networks. *Computers and Security* 109 (2021). doi:10.1016/j.cose.2021.102367

[19] Anja Feldmann. 2000. *Characteristics of TCP Connection Arrivals*. John Wiley Sons, Ltd, Chapter 15, 367–399. doi:10.1002/047120644X.ch15 arXiv:https://onlinelibrary.wiley.com/doi/pdf/10.1002/047120644X.ch15

[20] Anna Förster and Amy L. Murphys. 2007. FROMS: Feedback routing for optimizing multiple sinks in WSN with reinforcement learning. In *Proceedings of the 2007 International Conference on Intelligent Sensors, Sensor Networks and Information Processing, ISSNIP*. doi:10.1109/ISSNIP.2007.4496872

[21] Amir Globerson and Sam Roweis. 2006. Nightmare at test time: Robust learning by feature deletion. In *ACM International Conference Proceeding Series*, Vol. 148. doi:10.1145/1143844.1143889

[22] Ian Goodfellow, Patrick McDaniel, and Nicolas Papernot. 2018. Making machine learning robust against adversarial inputs: Such inputs distort how machine-learningbased systems are able to function in the world as it is. *Commun. ACM* 61 (2018). Issue 7. doi:10.1145/3134599

[23] Ian J. Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. 2014. Generative adversarial nets. In *Advances in Neural Information Processing Systems*, Vol. 3. Issue January. doi:10.1007/978-3-658-40442-0_9

[24] Ian J. Goodfellow, Jonathon Shlens, and Christian Szegedy. 2015. Explaining and harnessing adversarial examples. In *3rd International Conference on Learning Representations, ICLR 2015 - Conference Track Proceedings*.

[25] Dongqi Han, Zhiliang Wang, Ying Zhong, Wenqi Chen, Jiahai Yang, Shuqiang Lu, Xingang Shi, and Xia Yin. 2021. Evaluating and Improving Adversarial Robustness of Machine Learning-Based Network Intrusion Detectors. *IEEE Journal on Selected Areas in Communications* 39 (2021). Issue 8. doi:10.1109/JSAC.2021.3087242

[26] Mohammad J. Hashemi, Greg Cusack, and Eric Keller. 2019. Towards evaluation of NIDSs in adversarial setting. In *Big-DAMA 2019 - Proceedings of the 3rd ACM CoNEXT Workshop on Big DAta, Machine Learning and Artificial Intelligence for Data Communication Networks, Part of CoNEXT 2019*. doi:10.1145/3359992.3366642

[27] Minhao Jin and Maria Apostolaki. 2024. PANTS: Practical Adversarial Network Traffic Samples against ML-powered Networking Classifiers. (9 2024). https://arxiv.org/abs/2409.04691v1

[28] QUINLAN J.R. 1986. Induction of Decision Trees. *Machine Learning* (1986).

[29] Diederik P. Kingma and Max Welling. 2019. An Introduction to Variational Autoencoders. 307–392 pages. doi:10.1561/2200000056

[30] Franck Le, Mudhakar Srivatsa, Raghu Ganti, and Vyas Sekar. 2022. Rethinking Data-driven Networking with Foundation Models: Challenges and Opportunities. In *HotNets 2022 - Proceedings of the 2022 21st ACM Workshop on Hot Topics in Networks*. doi:10.1145/3563766.3564109

[31] Mike Lewis, Yinhan Liu, Naman Goyal, Marjan Ghazvininejad, Abdelrahman Mohamed, Omer Levy, Ves Stoyanov, and Luke Zettlemoyer. 2020. BART: Denoising sequence-to-sequence pre-training for natural language generation, translation, and comprehension. In *Proceedings of the Annual Meeting of the Association for Computational Linguistics*. doi:10.18653/v1/2020.acl-main.703

[32] Haoyu Liu, Alec F. Diallo, and Paul Patras. 2023. Amoeba: Circumventing ML-supported Network Censorship via Adversarial Reinforcement Learning. *Proceedings of the ACM on Networking* 1 (2023). Issue CoNEXT3. doi:10.1145/3629131

[33] Jun Liu, Ibrahim Matta, and Mark Crovella. 2003. End-to-End Inference of Loss Nature in a Hybrid Wired/Wireless Environment. (02 2003).

[34] Warren S. McCulloch and Walter Pitts. 1943. A logical calculus of the ideas immanent in nervous activity. *The Bulletin of Mathematical Biophysics* 5 (1943). Issue 4. doi:10.1007/BF02478259

[35] Yisroel Mirsky, Tomer Doitshman, Yuval Elovici, and Asaf Shabtai. 2018. Kitsune: An Ensemble of Autoencoders for Online Network Intrusion Detection. In *25th Annual Network and Distributed System Security Symposium, NDSS 2018*. doi:10.14722/ndss.2018.23204

[36] M. Sajid Mushtaq, Brice Augustin, and Abdelhamid Mellouk. 2012. Empirical study based on machine learning approach to assess the QoS/QoE correlation. In *2012 17th European Conference on Network and Optical Communications, NOC 2012, 7th Conference on Optical Cabling and Infrastructure, OC and I 2012*. doi:10.1109/NOC.2012.6249939

[37] Thuy T. T. Nguyen, Grenville Armitage, Philip Branch, and Sebastian Zander. 2012. Timely and Continuous Machine-Learning-Based Classification for Interactive IP Traffic. *IEEE/ACM Transactions on Networking* 20 (2012). Issue 6. doi:10.1109/tnet.2012.2187305

[38] Nicolas Papernot, Patrick McDaniel, and Ian Goodfellow. 2016. Transferability in Machine Learning: from Phenomena to Black-Box Attacks using Adversarial Samples. (5 2016). http://arxiv.org/abs/1605.07277

[39] Alessandro Pellegrini, Pierangelo Di Sanzo, and Dimiter R. Avresky. 2015. A Machine Learning-Based Framework for Building Application Failure Prediction Models. In *Proceedings - 2015 IEEE 29th International Parallel and Distributed Processing Symposium Workshops, IPDPSW 2015*. doi:10.1109/IPDPSW.2015.110

[40] Fabio Pierazzi, Feargus Pendlebury, Jacopo Cortellazzi, and Lorenzo Cavallaro. 2020. Intriguing properties of adversarial ML attacks in the problem space. In *Proceedings - IEEE Symposium on Security and Privacy*, Vol. 2020-May. doi:10.1109/SP40000.2020.00073

[41] Alec Radford. 2018. Improving Language Understanding by Generative Pre-Training. *Homology, Homotopy and Applications* 9 (2018). Issue 1.

[42] Colin Raffel, Noam Shazeer, Adam Roberts, Katherine Lee, Sharan Narang, Michael Matena, Yanqi Zhou, Wei Li, and Peter J. Liu. 2020. Exploring the limits of transfer learning with a unified text-to-text transformer. *Journal of Machine Learning Research* 21 (2020).

[43] Pavan Venkata Sainadh Reddy, Yelleti Vivek, Gopi Pranay, and Vadlamani Ravi. 2023. Chaotic Variational Auto encoder-based Adversarial Machine Learning. (2023). arXiv:2302.12959 [cs.LG] https://arxiv.org/abs/2302.12959

[44] Benjamin I.P. Rubinstein, Blaine Nelson, Ling Huang, Anthony D. Joseph, Shing Hon Lau, Satish Rao, Nina Taft, and J. D. Tygar. 2009. Antidote: Understanding and defending against poisoning of anomaly detectors. In *Proceedings of the ACM SIGCOMM Internet Measurement Conference, IMC*. doi:10.1145/1644893.1644895

[45] Iman Sharafaldin, Arash Habibi Lashkari, and Ali A. Ghorbani. 2018. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *ICISSP 2018 - Proceedings of the 4th International Conference on Information Systems Security and Privacy*, Vol. 2018-January. doi:10.5220/0006639801080116

[46] Jacob Steinhardt, Pang Wei Koh, and Percy Liang. 2017. Certified defenses for data poisoning attacks. In *Advances in Neural Information Processing Systems*, Vol. 2017-December.

[47] Peishuai Sun, Shuhao Li, Jiang Xie, Hongbo Xu, Zhenyu Cheng, and Rong Yang. 2023. GPMT: Generating practical malicious traffic based on adversarial attacks with little prior knowledge. *Computers and Security* 130 (2023). doi:10.1016/j.cose.2023.103257

[48] Christian Szegedy, Wojciech Zaremba, Ilya Sutskever, Joan Bruna, Dumitru Erhan, Ian Goodfellow, and Rob Fergus. 2014. Intriguing properties of neural networks. In *2nd International Conference on Learning Representations, ICLR 2014 - Conference Track Proceedings*.

[49] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Łukasz Kaiser, and Illia Polosukhin. 2017. Attention is all you need. In *Advances in Neural Information Processing Systems*, Vol. 2017-December.

[50] Christopher J. C. H. Watkins and Peter Dayan. 1992. Q-learning. *Machine Learning* 8 (1992). Issue 3-4. doi:10.1007/bf00992698

[51] Huang Xiao, Battista Biggio, Gavin Brown, Giorgio Fumera, Claudia Eckert, and Fabio Roli. 2015. Is feature selection secure against training data poisoning?. In *32nd International Conference on Machine Learning, ICML 2015*, Vol. 2.

[52] Yucheng Yin, Zinan Lin, Minhao Jin, Giulia Fanti, and Vyas Sekar. 2022. Practical GAN-based synthetic IP header trace generation using NetShare. In *SIGCOMM 2022 - Proceedings of the ACM SIGCOMM 2022 Conference*. doi:10.1145/3544216.3544251

[53] Edmund S. Yu and C. Y.Roger Chen. 1993. Traffic prediction using neural networks. In *IEEE Global Telecommunications Conference*, Vol. 2. Publ by IEEE, 991–995. doi:10.1109/glocom.1993.318226

[54] Jiawei Zhou, Woojeong Kim, Zhiying Xu, Alexander M. Rush, and Minlan Yu. 2024. NetFlowGen: Leveraging Generative Pre-training for Network Traffic Dynamics. arXiv:2412.20635 [cs.LG] https://arxiv.org/abs/2412.20635

[55] Bolor-Erdene Zolbayar, Ryan Sheatsley, Patrick McDaniel, Michael J. Weisman, Sencun Zhu, Shitong Zhu, and Srikanth Krishnamurthy. 2022. Generating Practical Adversarial Network Traffic Flows Using NIDSGAN. (3 2022). https://arxiv.org/abs/2203.06694v1